

Advanced Functional Programming - Final Project (Rust)

Elena Dumitrache (5812852) and Cosmin Visan (5710820)

Delft University of Technology, Delft, Netherlands

June 2026

Introduction

This project implements a small programming language with an ownership-based type system. The main goal is to reject programs that could use moved values, create invalid borrows, or mutate data in an unsafe way. The design is inspired by Rust's ownership model [1], but the language is much smaller and only contains the features needed for this project.

The language supports integers, booleans, a simple `Light` type, variables, mutable variables, assignments, blocks, conditionals, while loops, functions, lists, pairs, result values, pattern matching, references, mutable references, explicit lifetime functions, and spawn blocks. Programs are first parsed, then type-checked, and only then interpreted. This means that most errors are caught before the interpreter runs. The interpreter still performs runtime checks for errors that cannot be determined only from types. Invalid list indices and division by zero produce runtime errors instead of crashing the interpreter. A complete program must end with an expression, which determines its final type and runtime value.

The implementation is split into separate modules. The parser is generated from the grammar. The type checker is implemented in `TypeCheck.Expr`, `TypeCheck.Stmt`, `TypeCheck.Merge`, and `TypeCheck.Prog`. The interpreter follows the same structure in `Interp.Expr`, `Interp.Stmt`, and `Interp.Prog`. The context information is stored in `Context`, and nested scopes are handled by `ScopeStack`. This separation made the project easier to understand because type checking and runtime evaluation have similar shapes, but different responsibilities. The type checker checks the implemented type, mutability, ownership, and borrowing conditions. The interpreter then executes programs that were accepted by the checker.

1 Design Decisions

1.1 Rust-like ownership

The most important design decision was to make the language use an ownership model similar to Rust. Some values are copyable, and some values are not.

Integers, booleans, unit values, and immutable references are copyable, so using them does not remove them from their variable. `Result` values are copyable exactly when their inner type is copyable, and pairs are copyable exactly when both component types are copyable. Values such as `Light`, lists, and mutable references are non-copyable. When a non-copyable value is used, it is considered moved.

For example, this program is rejected:

Listing 1: Using a moved value

```
let a = Red;
let b = a;
a
```

After `let b = a`, the value owned by `a` has moved to `b`. Therefore, using `a` again is not allowed.

On the other hand, integers can be copied:

Listing 2: Copyable values can be reused

```
let x = 3;
let y = x;
x + y
```

This is valid because `int` is copyable. The type checker uses the helper function `isCopyable` to decide whether a value should be copied or moved.

1.2 Variables store more than just a type

A normal simple type checker only stores information like `x : int`. This project needs more information because of ownership and borrowing. Therefore, each variable in the type checker is stored using `VarInfo`. It remembers the variable type, whether it is mutable, whether it still owns its value, how many immutable borrows point to it, how many mutable borrows point to it, and whether the variable itself is a reference borrowing from another variable.

Conceptually, the variable context stores information like this:

$$x \mapsto (\tau, mut, owned, borrows, mutBorrows, borrowOf)$$

This is useful because the checker can reject examples like this:

Listing 3: Moving a borrowed value is rejected

```
let x = Red;
let r = &x;
let y = x;
* r
```

The value `x` cannot be moved while `r` still borrows it.

1.3 Mutable and immutable variables

The language distinguishes between immutable and mutable variables. Immutable variables cannot be assigned a new value. Mutable variables can be

updated. For an ordinary assignment, or an assignment from another variable, the destination cannot currently be borrowed. Direct assignments such as `r = &x` and `r = &mut x` are handled by separate cases, which release the old borrow and record the new borrow source.

Listing 4: Mutable assignment

```
let mut x = 1;
x = 2;
x
```

This returns 2. The following program is rejected because `x` is immutable:

Listing 5: Invalid assignment to immutable variable

```
let x = 1;
x = 2;
x
```

For an ordinary assignment, the type checker checks that the new expression has the same type as the destination. The expression is checked first, any old borrow held by the destination is then released, and the destination receives ownership of the new value. Direct borrow assignments and assignments from another variable use specialised cases.

1.4 Blocks and scopes

Blocks introduce a new scope. Variables declared inside a block disappear after the block ends, but assignments to variables from an outer scope still matter.

Listing 6: Block scope

```
let mut x = 0;
{
    let y = 5;
    x = y;
};
x
```

This returns 5. The variable `y` cannot be used outside the block. This is implemented using `ScopeStack`, where a new scope is pushed when entering a block and popped when leaving it.

1.5 Ownership across control flow

Ownership has to be tracked across branches, match arms, and loops, not only inside a single block.

In an `if-else`, the condition is checked first. The context produced after checking the condition is then saved. Both branches are checked separately from this same saved context, because only one of them will run.

After both branches have been checked, their resulting contexts are combined. A variable is considered owned after the `if-else` only if it is still owned after both branches. If either branch moves the value, the variable is treated as moved after the conditional.

For borrows, the checker keeps the larger borrow count from the two branches. This means that if a value may still be borrowed after either branch, it is treated as borrowed after the conditional. If the branches record different borrow sources for the same reference variable, the source is cleared because the checker cannot determine which source will be used at runtime.

An if statement without an else is handled in a similar way. The context after checking its body is combined with the context produced after checking the condition, because the condition may be false and the body may not run.

Therefore, the following program is accepted because the value is not used after the branches:

Listing 7: Moving a value in both conditional branches

```
fn consume(x : Light) -> () {};  
  
let x = Red;  
  
if true {  
    consume(x)  
} else {  
    consume(x)  
};  
  
0
```

However, using x after a branch that may consume it is rejected:

Listing 8: Using a value that may have been moved

```
fn consume(x : Light) -> () {};  
  
let x = Red;  
  
if true {  
    consume(x)  
} else {  
    0  
};  
  
x
```

A while loop may execute more than once. The type checker first checks the condition and then checks one execution of the loop body. The body is not allowed to move a non-copyable variable that was still owned after the condition was checked, because a later iteration could attempt to reuse the moved value:

Listing 9: Moving an outer variable inside a loop

```
fn consume(x : Light) -> () {};  
  
let x = Red;  
let mut i = 0;  
  
while i < 2 {  
    consume(x);  
}
```

```

    i = i + 1
};

0

```

After the loop body is checked, its resulting context is combined with the context produced after checking the condition. This represents the possibility that the condition is false and the body does not execute.

A value created inside the loop body can still be moved freely, because a fresh value is created on every iteration.

1.6 Scope stack versus a flat environment

One design choice was how to store variables from different scopes. A simpler option would have been to use one flat map that stores every variable currently known by the checker. However, this would make nested blocks and variable shadowing harder to handle.

For example:

Listing 10: Variable shadowing

```

let x = 1;
{
    let x = 2;
    x;
};
x

```

Inside the block, `x` should refer to 2. After leaving the block, it should refer to 1 again. With a flat environment, the old value of `x` would need to be saved before entering the block and restored afterwards. Variables created only inside the block would also need to be found and removed manually.

Instead, the implementation uses a `ScopeStack`, which is a list of maps. The first map represents the current scope, while the remaining maps represent the outer scopes. Entering a block pushes a new empty map onto the stack, and leaving the block removes it. Variable lookup starts from the current scope and continues towards the outer scopes.

The advantage of this approach is that nested scopes and variable shadowing are handled naturally. Removing a scope is also simple because the complete top map can be removed at once. The trade-off is that variable lookup may need to search through several maps instead of performing one lookup in a single flat map. The implementation accepts this small extra lookup cost because it makes scope handling clearer and less error-prone.

1.7 Borrowing

The language supports immutable borrows and mutable borrows.

An immutable borrow gives read access:

Listing 11: Immutable borrow

```

let x = Red;
let r = &x;

```

```
*r
```

A mutable borrow gives write access through dereferencing:

Listing 12: Mutable borrow

```
let mut x = 1;
let r = &mut x;
*r = 5;
x
```

This returns 5. Passing `&x` or `&mut x` directly as a function argument checks the borrow against the counters already recorded for `x`. Therefore, `f(&mut x)` is rejected when `x` is already borrowed elsewhere. These temporary argument borrows are validated but are not added to the context while the remaining function arguments are checked.

For borrows that are stored in variables and recorded in the context, the type checker allows several immutable borrows, but requires a mutable borrow to be exclusive. This means the following program is rejected:

Listing 13: Mutable borrow must be exclusive

```
let mut x = Red;
let r1 = &x;
let r2 = &mut x;
* r1
```

The mutable borrow is not allowed because `x` is already immutably borrowed.

1.8 Copying and moving references

An immutable reference is copyable, but a mutable reference is not. This matters when a reference is stored in a new variable. The new reference binding may itself be immutable or mutable, depending on whether it is introduced using `let` or `let mut`.

Listing 14: Copying an immutable reference

```
let x = Red;
let r = &x;
let s = r;
0
```

Here, `r` is an immutable borrow of `x`. Copying it into `s` does not end the borrow. Instead, `x` is now borrowed by both `r` and `s`, so `x` still cannot be moved while either reference is in use.

A mutable reference behaves differently:

Listing 15: Moving a mutable reference

```
let mut x = Red;
let b = &mut x;
let c = b;
0
```

A mutable reference cannot be copied, so the borrow is moved from `b` to `c`. Using `b` afterwards is rejected in the same way as using any other moved value.

When an existing reference variable is assigned from another reference variable, the old borrow held by the destination is released first. An immutable reference is then copied, while a mutable reference is moved and the source reference loses ownership.

1.9 Non-lexical lifetimes

A stricter implementation would keep every borrow active until the end of the block in which the reference was created. This project releases some borrows earlier by checking whether a reference will still be used.

After each statement, the type checker looks at the statements that remain in the current block. If a reference variable is not mentioned again, its borrow is released. This means that the original variable can be used or modified again before the end of the block.

The checker performs this analysis for reference variables declared in the current scope. A reference declared in an outer scope is checked again when the checker returns to that outer scope.

This allows programs like this:

Listing 16: Borrow ends after last use

```
let mut x = Red;
let r = &x;
*r;
x = Green;
x
```

The borrow `r` is no longer used after `*r`, so the assignment to `x` can be accepted.

1.10 Lists

Lists are supported for copyable element types. The language has list literals, indexing, assignment to an index, push, insert, and remove.

Listing 17: List operations

```
let mut xs = [1, 2];
xs.push(3);
xs[0] = 10;
xs.insert(1, 5);
xs.remove(2);
xs[0]
```

Reading a list element, assigning to an index, inserting an element, and removing an element all check the index at runtime. A negative or out-of-range index produces an error instead of crashing the interpreter. Inserting at the current length of the list is allowed and adds the value at the end.

The type checker allows a list to be read directly while it has immutable borrows, but not while it has an active mutable borrow. A list cannot be changed while it has any active immutable or mutable borrow.

Lists are non-copyable. This means that ownership of a list is moved when the list is assigned to another variable. List elements must have a copyable type. For example, lists of integers, booleans, immutable references, and copyable pairs are allowed. Lists containing `Light` values, other lists, or mutable references are rejected.

1.11 Result, pairs, and pattern matching

The language also supports `Result` values, pairs, and pattern matching. A `Result` value is copyable when the value stored inside it is copyable. A pair is copyable when both values inside the pair are copyable.

Listing 18: Pattern matching on `Result`

```
let r = Ok(5);
match r {
    Ok(x) => x,
    Err(e) => 0
}
```

The type checker checks that a match expression is exhaustive. For example, matching on a `Result` must cover both `Ok` and `Err`, unless a wildcard-like variable pattern is used. All match arms must also return the same type. Each match arm is checked separately from the same starting context. After all arms have been checked, their ownership and borrowing information is combined in the same way as the branches of an if-else.

1.12 Functions and lifetime functions

Functions can have multiple parameters, mutable parameters, statement bodies, and explicit return types. A function is added to the function context before its body is checked. This allows a function to call itself and to call functions that were declared earlier. Functions declared later are not available automatically, so general mutual recursion is not supported.

Listing 19: Function with statements

```
fn double(mut x: int) -> int {
    x = x * 2;
    x
}

double(21)
```

Normal functions are not allowed to have a reference as their direct return type. This includes immutable references, mutable references, and references with explicit lifetimes. This is because a normal function does not provide enough lifetime information to prove that the returned reference remains valid. The current check only examines the outermost return type. It does not recursively inspect references contained inside composite return types, such as pairs, `Result` values, or lists containing immutable references.

A lifetime function may return either a normal value or a reference. If its direct return type is a reference, that reference must use a lifetime declared in

the function header. Direct return types such as `&T` and `&mut T` are rejected because they do not specify a lifetime. As with normal functions, this check only examines the outermost return type and does not recursively inspect references nested inside composite return types.

Listing 20: Function with explicit lifetime

```
fn choose<'a>(x : &'a Light) -> &'a Light {
    x
};

let x = Red;
let r = choose(&x);
*r
```

When the outermost type of a parameter is a reference with an explicit lifetime, that lifetime must be declared in the function header. For example, a parameter of type `&'b int` is rejected when only `'a` was declared. Plain reference parameters such as `&int` and `&mut int` do not contain an explicit lifetime and are also accepted by the current checker. The checker does not recursively search for lifetime annotations nested inside composite parameter types such as pairs, lists, or `Result` values.

A reference returned by a lifetime function must be stored immediately using `let` or `let mut`. It cannot be used directly as a larger expression. When a lifetime function returns an immutable reference, the checker finds the first immutable reference parameter that uses the same lifetime. The matching argument must be written directly as `&x`.

When the function returns a mutable reference, the checker finds the first mutable reference parameter that uses the same lifetime. In this case, the matching argument must be written as `&mut x`. The newly created reference variable is then recorded as borrowing from `x`.

At runtime, a function stores its parameters and body, but it does not store a copy of the variable environment from the place where it was declared. Variables that are not parameters are therefore looked up using the scope stack that is active when the function is called.

1.13 Spawn blocks

The language includes a `spawn` construct. The current interpreter does not start a new thread for a spawn block. Instead, the block is executed immediately using the current runtime environment. Before accepting the block, the type checker collects the identifiers mentioned inside it. If one of those identifiers refers to a variable from an outer scope, that variable must have a copyable type.

Listing 21: Safe spawn capture

```
let x = 3;
spawn {
    let y = x + 1;
};
x
```

This example is accepted because `x` is an integer and integers are copyable. A non-copyable value such as `Light` is rejected when it is detected as a variable from the enclosing scope. The capture check is an approximation. It looks at identifiers mentioned in the body and compares them with variables from the outer scope. Because of this, an outer variable may still be detected even when a new local variable with the same name is declared inside the spawn block. Since the spawn block is executed synchronously in the current environment, assigning to an outer mutable copyable variable inside the block changes that outer variable.

1.13.1 Trade-off

We chose to execute spawn blocks synchronously because implementing real parallel execution would require a more complex runtime model. Each spawned block would need its own environment, and the interpreter would have to manage communication, shared mutable state, and the lifetime of captured variables across threads. The ownership checker would also need stronger rules to ensure that references and non-copyable values cannot be used unsafely by multiple threads.

Treating a spawn block as an ordinary block allowed us to reuse the existing scope, evaluation, and ownership-checking mechanisms. It also keeps execution deterministic and makes errors easier to test and reproduce. The trade-off is that the construct does not provide actual parallelism, and its capture check only approximates the restrictions that a fully concurrent implementation would require. Actual concurrency in the project is instead demonstrated by the logging system.

2 Typing Rules

This section presents the typing rules that use the extended ownership context. Equality and inequality are supported only for integers, booleans, and `Light` values. Ordered comparisons are supported only for integers. The type checker is stateful. Checking an expression can change the variable context. For example, using a non-copyable variable can move its value and change its ownership field. When this change is important, the judgment is written as

$$\Gamma, \Phi \vdash e : \tau \triangleright \Gamma', \Phi'.$$

This means that the expression has type τ and changes the contexts from Γ, Φ to Γ', Φ' .

The variable context Γ does not only store a type. It stores ownership and borrowing information too:

$$\Gamma(x) = (\tau, m, o, b, bm, src)$$

Here, τ is the type, m says whether the variable is mutable, o says whether it still owns its value, b is the number of active immutable borrows, bm is the number of active mutable borrows, and src is the variable borrowed from if this variable is a reference.

The function context Φ maps a function name to its complete parameter declarations and return type. For lifetime functions, it also stores the lifetimes declared in the function header.

2.1 Let-statements

Immutable and mutable let-statements differ only in the mutability flag stored in the context.

$$\frac{\Gamma, \Phi \vdash e : \tau \triangleright \Gamma_1, \Phi_1}{\Gamma, \Phi \vdash \text{let } x = e \triangleright \Gamma_1[x \mapsto (\tau, \text{false}, \text{true}, 0, 0, \perp)], \Phi_1} [\text{S-LETIMM}]$$

$$\frac{\Gamma, \Phi \vdash e : \tau \triangleright \Gamma_1, \Phi_1}{\Gamma, \Phi \vdash \text{let mut } x = e \triangleright \Gamma_1[x \mapsto (\tau, \text{true}, \text{true}, 0, 0, \perp)], \Phi_1} [\text{S-LETMUT}]$$

The expression form **let x = e in body** is handled separately. The checker first checks **e**, creates a temporary scope containing **x**, checks **body**, and then removes that scope. The new variable is stored without a recorded borrow source, so this form does not use the specialised borrow tracking of the statement-level let rules.

These rules describe ordinary declarations. Direct borrows, declarations from another variable, and calls whose result is immediately stored use specialised cases so that ownership and borrow-source information can be updated.

2.2 Variable use and ownership

Copyable values can be used without losing ownership.

$$\frac{\Gamma(x) = (\tau, m, o, b, bm, src) \quad \text{copy}(\tau)}{\Gamma, \Phi \vdash x : \tau \triangleright \Gamma, \Phi} [\text{E-VARCOPY}]$$

Non-copyable values can only be used if they still own their value and are not currently borrowed. After use, ownership is moved away.

$$\frac{\Gamma(x) = (\tau, m, \text{true}, 0, 0, src) \quad \neg \text{copy}(\tau)}{\Gamma, \Phi \vdash x : \tau \triangleright \Gamma[x.owned \mapsto \text{false}], \Phi} [\text{E-VARMOVE}]$$

This rule is the main affine ownership rule. It is what rejects using a value after it has been moved.

2.3 Assignment

Assignment is only allowed for mutable variables. The new expression must have the same type as the destination. If the destination previously stored a reference, its old borrow is released before the new value is stored.

$$\frac{\Gamma(x) = (\tau, \text{true}, o, 0, 0, src) \quad \Gamma, \Phi \vdash e : \tau \triangleright \Gamma_e, \Phi_e \quad \Gamma_1 = \text{release}(\Gamma_e, x)}{\Gamma, \Phi \vdash x = e \triangleright \Gamma_1[x.owned \mapsto \text{true}, x.src \mapsto \perp], \Phi_e} [\text{S-ASSIGN}]$$

$$\text{release}(\Gamma, x) = \begin{cases} \Gamma[y.b \mapsto y.b - 1] & \text{if } x.\text{src} = y \text{ and} \\ & x : \&\sigma \text{ or } x : \&^\ell\sigma, \\ \Gamma[y.b_m \mapsto y.b_m - 1] & \text{if } x.\text{src} = y \text{ and} \\ & x : \&\text{mut}\sigma \text{ or } x : \&\text{mut}^\ell\sigma, \\ \Gamma & \text{if } x.\text{src} = \perp, \text{ or if the} \\ & \text{recorded source is no longer in } \Gamma. \end{cases}$$

Assignments of the forms $\mathbf{r} = \&\mathbf{x}$, $\mathbf{r} = \&\text{mut } \mathbf{x}$, and $\mathbf{s} = \mathbf{r}$ use specialised cases. These cases first release the old borrow held by the destination. They then create a new borrow, copy an immutable reference, or move a mutable reference.

2.4 Control-flow statements

The function $\text{merge}(\Gamma_1, \Gamma_2)$ combines the information produced by two possible execution paths. A variable remains owned only when both paths keep it owned. For borrow counters, the larger value is kept. A borrow source is kept only when both paths record the same source. Variables that do not exist in both scope stacks are removed from the merged variable context. Function contexts are combined using map union. For a conditional expression, the condition is checked first. Both result expressions are then checked separately from the context produced by the condition.

$$\frac{\Gamma, \Phi \vdash \text{cond} : \text{bool} \triangleright \Gamma_0, \Phi_0 \quad \Gamma_0, \Phi_0 \vdash e_t : \tau \triangleright \Gamma_t, \Phi_t \quad \Gamma_0, \Phi_0 \vdash e_f : \tau \triangleright \Gamma_f, \Phi_f}{\Gamma, \Phi \vdash \text{if } \text{cond} \text{ then } e_t \text{ else } e_f : \tau \triangleright \text{merge}(\Gamma_t, \Gamma_f), \Phi_t \cup \Phi_f} \text{ [E-IF]}$$

An **if** statement without an **else** may execute its body or skip it. Therefore, the context after the body is combined with the context produced after checking the condition.

$$\frac{\Gamma, \Phi \vdash \text{cond} : \text{bool} \triangleright \Gamma_0, \Phi_0 \quad \Gamma_0, \Phi_0 \vdash \text{body} \triangleright \Gamma_b, \Phi_b}{\Gamma, \Phi \vdash \text{if } \text{cond} \text{ body} \triangleright \text{merge}(\Gamma_b, \Gamma_0), \Phi_b \cup \Phi_0} \text{ [S-IF]}$$

For an **if-else**, both branches are checked from the same starting context. Their resulting contexts are then combined.

$$\frac{\Gamma, \Phi \vdash \text{cond} : \text{bool} \triangleright \Gamma_0, \Phi_0 \quad \Gamma_0, \Phi_0 \vdash \text{tbody} \triangleright \Gamma_t, \Phi_t \quad \Gamma_0, \Phi_0 \vdash \text{fbody} \triangleright \Gamma_f, \Phi_f}{\Gamma, \Phi \vdash \text{if } \text{cond} \text{ tbody} \text{ else } \text{fbody} \triangleright \text{merge}(\Gamma_t, \Gamma_f), \Phi_t \cup \Phi_f} \text{ [S-IFELSE]}$$

A **while** loop may execute zero times or several times. A non-copyable variable that was owned before the loop may not be moved by the checked loop body.

$$\frac{\Gamma, \Phi \vdash \text{cond} : \text{bool} \triangleright \Gamma_0, \Phi_0 \quad \Gamma_0, \Phi_0 \vdash \text{body} \triangleright \Gamma_b, \Phi_b \quad \forall x. \text{owned}_{\Gamma_0}(x) \wedge \neg \text{copy}(\tau_x) \Rightarrow \text{owned}_{\Gamma_b}(x)}{\Gamma, \Phi \vdash \text{while } \text{cond} \text{ body} \triangleright \text{merge}(\Gamma_b, \Gamma_0), \Phi_b \cup \Phi_0} \text{ [S-WHILE]}$$

2.5 Immutable borrowing

An immutable borrow can be created if the borrowed value still exists and is not mutably borrowed.

Let m_r describe whether the new reference variable is mutable. It is *false* for `let r = ...` and *true* for `let mut r = ...`.

$$\frac{\Gamma(x) = (\tau, m, \text{true}, b, 0, \text{src})}{\Gamma, \Phi \vdash \text{let}_{m_r} r = \&x} \text{ [S-LETBORROW]}$$

$$\triangleright \Gamma[x.\text{borrows} \mapsto b + 1]$$

$$[r \mapsto (\&\tau, m_r, \text{true}, 0, 0, x)], \Phi$$

If the binding uses `let mut`, the reference variable is recorded as mutable instead, which is what lets it later be reassigned to a different borrow.

Several immutable borrows can exist at the same time. They only block moving or mutating the original value.

2.6 Mutable borrowing

A mutable borrow can only be created from a mutable variable. It is exclusive, so there must be no active immutable or mutable borrows.

$$\frac{\Gamma(x) = (\tau, \text{true}, \text{true}, 0, 0, \text{src})}{\Gamma, \Phi \vdash \text{let}_{m_r} r = \&\text{mut } x} \text{ [S-LETMUTBORROW]}$$

$$\triangleright \Gamma[x.\text{mutBorrows} \mapsto 1][r \mapsto (\&\text{mut } \tau, m_r, \text{true}, 0, 0, x)], \Phi$$

If the binding uses `let mut`, the reference variable is recorded as mutable instead, which is what lets it later be reassigned to a different borrow.

This rule enforces the idea that a mutable reference must be the only active reference to a value.

The next two rules describe creating a new variable from an existing reference variable. Let m_s describe whether the new variable is declared using `let` or `let mut`.

Let

$$\rho_I \in \{\&\tau, \&^\ell \tau\}$$

be an immutable reference type, and let

$$\rho_M \in \{\&\text{mut } \tau, \&\text{mut}^\ell \tau\}$$

be a mutable reference type.

$$\frac{\Gamma(r) = (\rho_I, m_r, \text{true}, b, bm, y) \quad \Gamma(y) = (\tau, m_y, o_y, b_y, 0, \text{src}_y)}{\Gamma, \Phi \vdash \text{let}_{m_s} s = r \triangleright \Gamma[y.\text{borrows} \mapsto b_y + 1][s \mapsto (\rho_I, m_s, \text{true}, 0, 0, y)], \Phi} \text{ [S-LETCOPYREF]}$$

$$\frac{\Gamma(r) = (\rho_M, m_r, \text{true}, b, bm, y)}{\Gamma, \Phi \vdash \text{let}_{m_s} s = r} \text{ [S-LETMOVEREF]}$$

$$\triangleright \Gamma[r.\text{owned} \mapsto \text{false}, r.\text{src} \mapsto \perp][s \mapsto (\rho_M, m_s, \text{true}, 0, 0, y)], \Phi$$

If the source reference has no recorded borrow source, the new reference also has no recorded source and no borrow counter is changed. The same copy-or-move behaviour is used for `s = r` between existing variables. In that case, the old borrow held by `s` is released before the new reference is stored.

2.7 Dereferencing

Dereferencing a reference gives the type stored inside the reference. Let

$$R(\tau) = \{\&\tau, \&mut\ \tau, \&\ell\tau, \&mut\ell\tau\}.$$

$$\frac{\Gamma, \Phi \vdash e : \rho \triangleright \Gamma_1, \Phi_1 \quad \rho \in R(\tau)}{\Gamma, \Phi \vdash *e : \tau \triangleright \Gamma_1, \Phi_1} \text{ [E-DEREF]}$$

When the operand of $*$ is a reference variable, it is treated as a place expression. Dereferencing reads through it without consuming the reference variable itself.

$$\frac{\Gamma(r) = (\rho, m, true, b, bm, src) \quad \rho \in R(\tau)}{\Gamma, \Phi \vdash *r : \tau \triangleright \Gamma, \Phi \quad (r \text{ is not consumed})} \text{ [E-DEREFPLACE]}$$

Writing through a dereference is only allowed for mutable references.

$$\frac{\Gamma(r) = (\rho_M, m, true, b, bm, src) \quad \Gamma, \Phi \vdash e : \tau \triangleright \Gamma_1, \Phi_1}{\Gamma, \Phi \vdash *r = e \triangleright \Gamma_1, \Phi_1} \text{ [S-DEREFASSIGN]}$$

2.8 Lists

A list literal must have at least one element, all elements must have the same type, and the element type must be copyable.

$$\frac{\Gamma, \Phi \vdash e_1 : \tau \triangleright \Gamma_1, \Phi_1 \quad \Gamma_{i-1}, \Phi_{i-1} \vdash e_i : \tau \triangleright \Gamma_i, \Phi_i \quad \text{for } 2 \leq i \leq n \quad copy(\tau)}{\Gamma, \Phi \vdash [e_1, \dots, e_n] : [\tau] \triangleright \Gamma_n, \Phi_n} \text{ [E-LIST]}$$

Indexing a list requires an integer index and returns the element type.

$$\frac{\Gamma(x) = ([\tau], m, true, b, 0, src) \quad \Gamma, \Phi \vdash i : int \triangleright \Gamma_1, \Phi_1}{\Gamma, \Phi \vdash x[i] : \tau \triangleright \Gamma_1, \Phi_1} \text{ [E-INDEX]}$$

Mutating a list requires the list variable to be mutable.

$$\frac{\Gamma(x) = ([\tau], true, true, 0, 0, src) \quad \Gamma, \Phi \vdash i : int \triangleright \Gamma_1, \Phi_1 \quad \Gamma_1, \Phi_1 \vdash e : \tau \triangleright \Gamma_2, \Phi_2}{\Gamma, \Phi \vdash x[i] = e \triangleright \Gamma_2, \Phi_2} \text{ [S-INDEXASSIGN]}$$

The rules for **push**, **insert**, and **remove** use the same requirements: the list must be mutable, must still own its value, and must not currently be borrowed. The **push** operation checks the type of the new value. The **insert** operation checks both the integer index and the type of the new value. The **remove** operation only checks that the index has type **int**. Any ownership or borrowing changes produced while checking their index or value expressions are kept after the operation.

2.9 Functions

A normal function can have several parameters and a block body. The function is added to the function context before its body is checked. This allows the function to call itself and to call functions that were declared earlier. For each parameter x_i , let m_i be *true* when it was declared with *mut*, and *false* otherwise. The parameter context is

$$\Gamma_p = \text{push}(\Gamma) [x_i \mapsto (\tau_i, m_i, \text{true}, 0, 0, \perp)]_{i=1}^n.$$

The operation $\text{push}(\Gamma)$ creates a new scope for the function parameters. This allows a parameter to shadow a variable from an outer scope without replacing that outer binding.

The predicate $\text{isRef}(\tau)$ is true for all four reference forms:

$$\text{isRef}(\tau) \quad \text{holds for} \quad \&\sigma, \&\text{mut } \sigma, \&^\ell\sigma, \&\text{mut}^\ell\sigma.$$

Let

$$\Phi' = \Phi[f \mapsto (\text{params}, \text{retTy})].$$

The operation

$$\text{checkBody}(\Gamma_p, \Phi', \text{body}, \text{retTy}) = (\Gamma_{\text{out}}, \Phi_{\text{out}})$$

represents checking the function body, releasing borrows from the parameter scope, and then removing that scope. For return type $()$, all statements are checked but no final unit expression is required. For every other return type, the body must end with an expression of the declared type.

$$\frac{\neg \text{isRef}(\text{retTy}) \quad \text{checkBody}(\Gamma_p, \Phi', \text{body}, \text{retTy}) = (\Gamma_{\text{out}}, \Phi_{\text{out}})}{\Gamma, \Phi \vdash \text{fn } f(\text{params}) \rightarrow \text{retTy} \quad \text{body} \triangleright \Gamma_{\text{out}}, \Phi_{\text{out}}} \text{ [S-FUN]}$$

Checking the body can update ownership information for variables from outer scopes because those scopes remain below the parameter scope. Function declarations encountered inside the body also remain in the resulting function context. For a lifetime function, define

$$\text{validLifetimeReturn}(L, \text{retTy})$$

as follows:

$$\text{validLifetimeReturn}(L, \text{retTy}) \quad \text{holds when}$$

$$\begin{cases} \ell \in L, & \text{if } \text{retTy} = \&^\ell\tau \text{ or } \&\text{mut}^\ell\tau, \\ \text{false}, & \text{if } \text{retTy} = \&\tau \text{ or } \&\text{mut } \tau, \\ \text{true}, & \text{if } \text{retTy} \text{ is not a reference type.} \end{cases}$$

Let

$$\Phi' = \Phi[f \mapsto (L, \text{params}, \text{retTy})].$$

$$\frac{\text{validLifetimeReturn}(L, \text{retTy}) \quad \text{lifetimes}(\text{params}) \subseteq L \quad \text{checkBody}(\Gamma_p, \Phi', \text{body}, \text{retTy}) = (\Gamma_{\text{out}}, \Phi_{\text{out}})}{\Gamma, \Phi \vdash \text{fn } f\langle L \rangle(\text{params}) \rightarrow \text{retTy} \quad \text{body} \triangleright \Gamma_{\text{out}}, \Phi_{\text{out}}} \text{ [S-FUNLT]}$$

Here, $lifetimes(params)$ contains only the explicit lifetime annotations that occur in the parameter types. Plain parameter types such as $\&T$ and $\&mut\ T$ contribute no lifetime to this set.

2.10 Pattern matching

A match expression first checks the expression being matched. Every pattern must be valid for the type of that expression, and the patterns must cover all possible cases.

Each match arm is checked separately from the same starting context Γ_0, Φ_0 . The first arm determines the result type τ . Every remaining arm must return the same type.

While an arm is checked, variables introduced by its pattern are added to a temporary scope. This scope is removed after the arm expression has been checked.

Let

$$\text{inferArm}(\Gamma_0, \Phi_0, \sigma, pat_1, e_1) = (\tau, \Gamma_1, \Phi_1)$$

represent checking the first arm and determining its result type. Let

$$\text{checkArm}(\Gamma_0, \Phi_0, \sigma, pat_i, e_i, \tau) = (\Gamma_i, \Phi_i) \quad \text{for } 2 \leq i \leq n$$

represent checking every remaining arm and requiring it to have type τ . Both operations first create a temporary scope. The pattern is then validated, and any variables introduced by it are added to that scope. The arm expression is checked next, after which the temporary scope is removed.

$$\frac{\begin{array}{c} \Gamma, \Phi \vdash e : \sigma \triangleright \Gamma_0, \Phi_0 \\ \text{exhaustive}(\sigma, pats) \end{array} \quad \begin{array}{c} \text{inferArm}(\Gamma_0, \Phi_0, \sigma, \\ pat_1, e_1) = (\tau, \Gamma_1, \Phi_1) \end{array} \quad \begin{array}{c} \text{checkArm}(\Gamma_0, \Phi_0, \sigma, pat_i, e_i, \tau) \\ = (\Gamma_i, \Phi_i), \quad 2 \leq i \leq n \end{array}}{\Gamma, \Phi \vdash \text{match } e \{ arms \} : \tau} \quad [\text{E-MATCH}]$$

$$\triangleright \text{mergeAll}(\Gamma_1, \dots, \Gamma_n), \bigcup_{i=1}^n \Phi_i$$

The operation $\text{mergeAll}(\Gamma_1, \dots, \Gamma_n)$ applies the previously defined merge operation repeatedly until the contexts produced by all match arms have been combined.

The predicate $\text{exhaustive}(\sigma, pats)$ means that the patterns in $pats$ cover every possible value of type σ .

For **Light**, the patterns must cover **Red**, **Yellow**, and **Green**. For **Result**, they must cover **Ok** and **Err**. For pairs, one pair pattern is enough. A variable pattern works as a wildcard.

2.11 Spawn blocks

A spawn block may only use enclosing variables whose types are copyable. The checker first collects the identifiers mentioned in the body and checks which of them resolve to variables already present in the enclosing context.

$$\text{captured}_\Gamma(\text{body}) = \text{mentioned}(\text{body}) \cap \text{dom}(\Gamma)$$

$$\frac{\text{captured}_\Gamma(\text{body}) = \{x_1, \dots, x_n\} \quad \text{copy}(\text{type}_\Gamma(x_i)) \quad \text{for all } i \quad \Gamma, \Phi \vdash \text{body} \triangleright \Gamma', \Phi'}{\Gamma, \Phi \vdash \text{spawn } \text{body} \triangleright \Gamma', \Phi'} \text{ [S-SPAWN]}$$

The set $\text{captured}_\Gamma(\text{body})$ is an approximation. It contains identifiers mentioned in the body that also resolve to variables in the enclosing context. Because the check is performed before the local spawn scope is created, an outer variable may still be detected when a local variable with the same name shadows it. The interpreter executes the spawn block synchronously as an ordinary block. It does not create a new thread.

3 Techniques

3.1 Persistent purely functional data structure

The main purely functional data structure is `ScopeStack`. It is implemented as a list of maps:

Listing 22: Scope stack definition

```
newtype ScopeStack a = ScopeStack [Map.Map Ident a]
```

The first map is the current scope, and the remaining maps are outer scopes. When entering a new block or function, the implementation calls `push`, which adds a new empty map to the front. When leaving the scope, it calls `pop`. Lookup starts from the current scope and continues outward, so shadowing works naturally.

For example:

Listing 23: Nested scopes

```
let x = 1;
{
  let x = 2;
  x;
};
x
```

Inside the block, `x` refers to the inner value. After the block, the inner scope is removed and `x` refers to the outer value again.

The structure is purely functional because its operations do not mutate an existing scope stack. Instead, they return a new version while the previous version remains available. The type checker actively uses this persistence when analysing control flow. For an `if-else`, it saves the context before checking the branches, checks the true branch, restores the same saved context, and then checks the false branch independently. The two resulting contexts are later merged. Similarly, when checking a `while` loop, the contexts before and after the body are kept at the same time and compared to detect moved variables. Expression-level conditionals and match arms use the same approach. Because the stack is built from immutable lists and maps, these saved versions can share unchanged data and do not require an explicit deep copy.

3.2 Lenses and traversals

The implementation uses simple Van Laarhoven lenses in `Context`. The lenses are used to access and update fields inside the type-checking and evaluation contexts without writing repeated record update code.

For example, the type checker updates the variable context like this:

Listing 24: Updating the type-checker variable context

```
modify (over tcVars (insertTop x info))
```

Here, `tcVars` focuses on the variable part of the type-checker context, and `over` updates that field. This is easier to read than repeatedly unpacking and rebuilding the whole context record.

Traversals are also used for non-lexical lifetimes. The function `releaseExpiredBorrows` uses `Map.traverseWithKey` together with `Const` to inspect the bindings in the top scope and collect references that are no longer used. Those borrows can then be released early. In other words, traversal is used to scan the current scope without manually writing a separate recursive loop.

3.3 Monad transformers

The type checker uses the following monad:

Listing 25: Type-checker monad

```
type Tc a = ExceptT String (State TcCtx) a
```

This combines two effects. `ExceptT String` allows the checker to stop with an error message, and `State TcCtx` carries the current type-checking context. This is useful because type checking constantly updates information about variables, ownership, borrowing, and functions.

The interpreter uses a similar monad:

Listing 26: Interpreter monad

```
type Eval a = ExceptT String (State EvalCtx) a
```

The difference is that `EvalCtx` stores runtime values instead of type information. The same structure is used: errors are handled with `ExceptT`, and the changing runtime environment is handled with `State`.

This made the code simpler because functions can report errors directly using `throwError`, while still reading and updating the context using `gets` and `modify`.

3.4 Concurrency

Concurrency is used in the logging system. The logger uses a `Chan` to receive log messages and an `MVar` to signal when the logger has finished. When logging starts, a background thread is created with `forkIO`. The main program can send messages into the channel, and the logger thread prints them.

The logger uses `Maybe LogMsg` in the channel. A normal log message is sent as `Just msg`. The value `Nothing` is used as a stop signal. When the logger receives `Nothing`, it writes to the `MVar` to tell the main program that it is done.

Conclusion

The project implements a small language with a Rust-inspired ownership type system. The most important part of the implementation is the type checker, which tracks not only types, but also mutability, ownership, and active borrows. This allows it to reject moved non-copyable values, many invalid assignments, conflicting tracked borrows, and non-copyable variables detected by the spawn capture analysis. The implementation uses the simplifying assumptions described earlier rather than implementing the complete Rust memory model.

The implementation uses a clear pipeline: source code is parsed into an AST (Abstract Syntax Tree), the AST is type-checked, and only programs accepted by the implemented checker are interpreted. Errors that depend on runtime values, such as invalid list indices and division by zero, are reported by the interpreter. The project also uses the required functional programming techniques: a purely functional scope stack, lenses and traversals for context updates, monad transformers for errors and state, and concurrency for the logger.

AI Usage Disclosure

Generative AI was used during the testing and review stages of this project to suggest additional test cases and identify edge cases that we might otherwise have overlooked. While evaluating these suggested tests, we found that one test involving an `if-else` statement did not behave according to the intended language semantics. The test used the same non-copyable value separately in both branches. Although only one branch can be executed at runtime, our original implementation type-checked the branches sequentially using the same evolving type-checker state. Consequently, after the value was marked as moved while checking the first branch, its use in the second branch was incorrectly rejected.

After examining the failing test and verifying the expected behaviour against our language semantics, we modified the implementation so that both branches are type-checked independently from the same initial context. The resulting type-checker states are then merged conservatively. Thus, generative AI helped us discover the issue by proposing an additional test case, while the analysis of the problem, the design of the solution, and its implementation and validation were carried out by us.

Generative AI tools were also used to help refine the wording of the report, mainly through rephrasing sentences and improving readability. The ideas, language design, experiments, technical decisions, and conclusions are entirely our own. AI-generated suggestions were never incorporated automatically, each suggestion was reviewed, verified, and adapted by us before being included. AI was therefore used as a supporting tool for improving test coverage, identifying possible edge cases, and refining the presentation of the text, rather than for producing the final implementation or determining the conclusions of the project.

References

- [1] S. Klabnik and C. Nichols, “Understanding Ownership,” in *The Rust Programming Language*, Second, San Francisco, CA, USA: No Starch Press, 2021, ISBN: 978-1-71850-310-6.